

Milestone #2 Data Summary

Author: Aaron Perez

Project Title: GM Simulator

Research Question Reminder

The primary research question is to forecast Major League Baseball game outcomes by analyzing individual player performance history, environmental conditions, and overall team health to estimate a team's probability of winning. The project utilizes historical MLB data spanning from the 2023 season to the current season. Incorporating multiple seasons provides a great sample size for evaluating player performance trends, environmental variables, and team characteristics across different venues and game scenarios. At the completion of the collection phase, the dataset will be frozen to ensure that feature engineering, model development, and evaluation are conducted on a consistent and reproducible version of the data. Furthermore, a backfill script was run to secure comprehensive information for the preceding season.

Data Inventory

Data Source

Data Source & Endpoint	API URL Pattern	License / Terms	Estimated Size / Row Count	Date Range	Refresh Policy	Warehouse Target Table	Primary Model Features Derived
Schedule Index	/api/v1/schedule?sportId=1&date={YYYY-MM-DD}	Open / Public Non-Commercial	~7,500 Rows / 2.1 MB	2023-03-30 to 2026-06-27	Backfill Loop / Daily Runner	public.games	Game outcomes, home field identification, doubleheader flags, date dimensions.
Live Telemetry Feed	/api/v1.1/game/{gamePk}/feed/live	Open / Public Non-Commercial	~2.3M Pitches / 1.8 GB	2023-03-30 to 2026-06-27	Backfill Loop / Daily Runner	public.statcast_pitch	Pitch velocity, spin rate, plate coordinates, launch angle, stadium wind splits, game temperature.

Boxscore Personnel	/api/v1/game/{gamePk}/boxscore	Open / Public Non-Commercial	~120 Rows / 16 KB	2023-03-30 to 2026-06-27	Backfill Loop / Daily Runner	public.umpires	Field umpire assignments (Home Plate umpire tendency matrices).
Transaction Wire	/api/v1/transactions?sportId=1&date={YYYY-MM-DD}	Open / Public Non-Commercial	~45,000 Rows / 12 MB	2023-03-30 to 2026-06-27	Backfill Loop / Daily Runner	public.roster_actions	Roster status moves (Called Up, Optioned, Injured List tracker) to gauge bullpen fatigue.
Master Venue Index	/api/v1/venues?sportId=1	Open / Public Non-Commercial	~60 Rows / 16 KB	Static (2026 Registry)	Static Initialization Phase	public.parks	Park names and primary venue structural identification mappings.
Franchise Registry	/api/v1/teams?sportId=1	Open / Public Non-Commercial	30 Rows / 8 KB	Static (2026 Align)	Static Initialization Phase	public.teams	Team names, corporate tracking codes, abbreviations, and geographic location designations.
Player Biography Registry	/api/v1/sports/1/playerlayers?season={YYYY}	Open / Public Non-Commercial	~4,200 Rows / 512 KB	2023-03-30 to 2026-06-27	Static Initialization & Daily JIT Callback	public.players	Player biographical constants (bats, throws, height, weight, birth date, MLB debut tracking).

Data Tables

Table Name	Source	License / Terms	Estimated Row Count / Size	Date Range	Refresh Policy	Location in Warehouse
public.parks	MLB Stats API (/venues)	Open/Public Non-Commercial	~60 Rows / 16 KB	Static (2023–2026)	Static Initialization	Production DB Dimension
public.teams	MLB Stats API (/teams)	Open/Public Non-Commercial	30 Rows / 8 KB	Static (2026 Align)	Static Initialization	Production DB Dimension
public.players	MLB Stats API (/sports/1/player/s)	Open/Public Non-Commercial	~4,200 Rows / 512 KB	2023-03-30 to 2026-06-27	Daily Transactional JIT	Production DB Dimension
public.umpires	MLB Stats API (/boxscore)	Open/Public Non-Commercial	~120 Rows / 16 KB	2023-03-30 to 2026-06-27	Dynamic JIT Lookup	Production DB Dimension
public.games	MLB Stats API (/schedule)	Open/Public Non-Commercial	~7,500 Rows / 2.1 MB	2023-03-30 to 2026-06-27	Continuous Loop	Production DB Fact

SQL Code Table Creation and SQL View Creation

```
-- games
CREATE TABLE public.games (
  game_pk integer NOT NULL,
  game_date date NOT NULL,
  season integer NOT NULL,
  game_type character varying(10) NULL,
  scheduled_start timestamp without time zone NULL,
  park_id integer NULL,
  home_team_id integer NULL,
  away_team_id integer NULL,
  home_score integer NULL,
  away_score integer NULL,
  day_night_type character varying(20) NULL,
  is_doubleheader boolean NULL,
  temperature_f integer NULL,
  sky_condition character varying(100) NULL,
  wind_speed_mph integer NULL,
  wind_direction character varying(50) NULL,
  home_plate_ump_id integer NULL,
  first_base_ump_id integer NULL,
  second_base_ump_id integer NULL,
  third_base_ump_id integer NULL,
  winning_team_id integer NULL,
  is_home_team_win boolean NULL
);

ALTER TABLE public.games
ADD CONSTRAINT games_pkey PRIMARY KEY (game_pk)

-- Parks
CREATE TABLE public.parks (
  park_id integer NOT NULL,
  park_name character varying(150) NULL,
  elevation integer NULL
);

ALTER TABLE public.parks
ADD CONSTRAINT parks_pkey PRIMARY KEY (park_id)

-- Players
CREATE TABLE public.players (
  player_id integer NOT NULL,
  full_name character varying(150) NULL,
  current_team_id integer NULL,
  position_code character varying(10) NULL,
  bats character varying(5) NULL,
  throws character varying(5) NULL,
  birth_date date NULL,
  height character varying(20) NULL,
  weight integer NULL,
  mlb_debut date NULL,
  is_active boolean NULL
);

ALTER TABLE public.players
ADD CONSTRAINT players_pkey PRIMARY KEY (player_id)

-- Starting Lineup
CREATE TABLE public.starting_lineups (
  id serial NOT NULL,
  game_pk integer NULL,
  team_id integer NULL,
  batting_order_slot integer NOT NULL,
  player_id integer NULL,
  field_position character varying(10) NULL,
  batting_side character varying(1) NULL
);
```

```
ALTER TABLE public.starting_lineups
ADD CONSTRAINT starting_lineups_pkey PRIMARY KEY (id)
```

```
-- Pitch by Pitch Data
```

```
CREATE TABLE public.statcast_pitches (
pitch_id bigserial NOT NULL,
game_pk integer NULL,
game_date date NULL,
plate_appearance_number integer NULL,
at_bat_number integer NULL,
pitch_number integer NULL,
inning integer NULL,
inning_half character varying(3) NULL,
outs integer NULL,
ball_count integer NULL,
strike_count integer NULL,
batter_id integer NULL,
pitcher_id integer NULL,
pitch_type character varying(10) NULL,
release_velocity numeric NULL,
release_spin_rate integer NULL,
release_extension numeric NULL,
release_pos_x numeric NULL,
release_pos_y numeric NULL,
release_pos_z numeric NULL,
vx0 numeric NULL,
vy0 numeric NULL,
vz0 numeric NULL,
ax numeric NULL,
ay numeric NULL,
az numeric NULL,
effective_speed numeric NULL,
plate_crossing_x numeric NULL,
plate_crossing_z numeric NULL,
sz_top numeric NULL,
sz_bot numeric NULL,
runner_on_first boolean NULL,
runner_on_second boolean NULL,
runner_on_third boolean NULL,
home_score integer NULL,
away_score integer NULL,
play_event text NULL,
play_description text NULL,
exit_velocity numeric NULL,
launch_angle numeric NULL,
hit_distance numeric NULL,
spray_angle numeric NULL,
hit_location_x numeric NULL,
hit_location_y numeric NULL,
expected_woba numeric NULL,
expected_slugging numeric NULL,
is_hard_hit boolean NULL,
is_sweet_spot boolean NULL
);
```

```
ALTER TABLE public.statcast_pitches
ADD CONSTRAINT statcast_pitches_pkey PRIMARY KEY (pitch_id)
```

```
-- Teams
```

```
CREATE TABLE public.teams (
team_id integer NOT NULL,
abbreviation character varying(10) NULL,
team_name character varying(100) NULL,
city character varying(100) NULL,
nickname character varying(100) NULL,
league character varying(50) NULL,
division character varying(100) NULL,
team_code character varying(10) NULL,
location_name text NULL
);
```

```

ALTER TABLE public.teams
ADD CONSTRAINT teams_pkey PRIMARY KEY (team_id)

-- Injury/ Player Transaction table
CREATE TABLE public.transactions (
  id serial NOT NULL,
  player_id integer NULL,
  transaction_date date NOT NULL,
  transaction_type character varying(100) NULL,
  from_team_id integer NULL,
  to_team_id integer NULL,
  injury_status text NULL
);

ALTER TABLE public.transactions
ADD CONSTRAINT transactions_pkey PRIMARY KEY (id)

-- Umpires Data
CREATE TABLE public.umpires (
  umpire_id integer NOT NULL,
  umpire_name character varying(150) NULL
);

ALTER TABLE public.umpires
ADD CONSTRAINT umpires_pkey PRIMARY KEY (umpire_id)

-- Bullpen Usage View
CREATE OR REPLACE VIEW "public"."view_bullpen_usage" AS
SELECT
  pitcher_id,
  game_pk,
  count(pitch_number) AS total_pitch_count,
  count(DISTINCT at_bat_number) AS batters_faced,
  round(
    count(
      CASE
        WHEN play_event = ANY (
          ARRAY[
            'out'::text,
            'field_out'::text,
            'strikeout'::text,
            'grounded_into_double_play'::text
          ]
        ) THEN 1
        ELSE NULL::integer
      END
    )::numeric / 3.0,
    1
  ) AS innings_pitched
FROM
  statcast_pitches
GROUP BY
  pitcher_id,
  game_pk;

-- Pitcher's Favorite Pitches
CREATE OR REPLACE VIEW "public"."view_pitch_arsenal" AS
SELECT
  pitcher_id,
  pitch_type,
  count(*) AS total_pitches_thrown,
  round(avg(release_velocity), 1) AS avg_velocity_mph,
  round(avg(release_spin_rate), 0) AS avg_spin_rate_rpm,
  round(
    count(
      CASE
        WHEN play_description = ANY (
          ARRAY[
            'swinging_strike'::text,

```

```

        'swinging_strike_blocked'::text,
        'foul_tip'::text
    ]
) THEN 1
ELSE NULL::integer
END
)::numeric * 100.0 / NULLIF(
count(
CASE
WHEN play_description = ANY (
ARRAY[
'swinging_strike'::text,
'swinging_strike_blocked'::text,
'foul_tip'::text,
'foul'::text,
'hit_into_play'::text
]
) THEN 1
ELSE NULL::integer
END
),
0
)::numeric,
1
) AS whiff_percentage
FROM
statcast_pitches
WHERE
pitch_type IS NOT NULL
GROUP BY
pitcher_id,
pitch_type;

-- Plate Apperance View
CREATE OR REPLACE VIEW "public"."view_plate_appearances" AS
SELECT
game_pk,
batter_id,
pitcher_id,
at_bat_number,
count(pitch_number) AS pitch_sequence_count,
max(play_event) AS at_bat_outcome,
max(play_description) AS final_pitch_description
FROM
statcast_pitches
GROUP BY
game_pk,
batter_id,
pitcher_id,
at_bat_number;

-- Average Pitches thrown by Pitcher
CREATE OR REPLACE VIEW "public"."view_player_features" AS
SELECT
pitcher_id,
game_pk,
avg(total_pitch_count) OVER (
PARTITION BY
pitcher_id
ORDER BY
game_pk ROWS BETWEEN 3 PRECEDING
AND 1 PRECEDING
) AS rolling_avg_pitches_thrown
FROM
view_bullpen_usage;

-- Weather Impact
CREATE OR REPLACE VIEW "public"."view_prediction_dataset" AS
SELECT
g.game_pk,

```



```

g.game_date,
g.home_team_id,
g.away_team_id,
g.temperature_f,
g.wind_speed_mph,
p.elevation AS park_elevation,
hf.wins_last_10 AS home_team_wins_last_10,
af.wins_last_10 AS away_team_wins_last_10,
CASE
  WHEN g.is_home_team_win = true THEN 1
  ELSE 0
END AS target_home_team_win
FROM
games g
LEFT JOIN parks p ON g.park_id = p.park_id
LEFT JOIN view_team_features hf ON g.game_pk = hf.game_pk
AND g.home_team_id = hf.team_id
LEFT JOIN view_team_features af ON g.game_pk = af.game_pk
AND g.away_team_id = af.team_id;

```

-- How Well a team been performing

```

CREATE OR REPLACE VIEW "public"."view_team_features" AS
SELECT
team_id,
game_pk,
game_date,
sum(is_win) OVER (
  PARTITION BY
    team_id
  ORDER BY
    game_date ROWS BETWEEN 5 PRECEDING
    AND 1 PRECEDING
) AS wins_last_5,
sum(is_win) OVER (
  PARTITION BY
    team_id
  ORDER BY
    game_date ROWS BETWEEN 10 PRECEDING
    AND 1 PRECEDING
) AS wins_last_10,
avg(runs_scored) OVER (
  PARTITION BY
    team_id
  ORDER BY
    game_date ROWS BETWEEN 10 PRECEDING
    AND 1 PRECEDING
) AS avg_runs_scored_last_10
FROM
view_team_games;

```

-- Team verse home and awya

```

CREATE OR REPLACE VIEW "public"."view_team_games" AS
SELECT
games.game_pk,
games.game_date,
games.season,
games.home_team_id AS team_id,
games.home_score AS runs_scored,
games.away_score AS runs_allowed,
CASE
  WHEN games.is_home_team_win = true THEN 1
  ELSE 0
END AS is_win
FROM
games
UNION ALL
SELECT
games.game_pk,
games.game_date,
games.season,

```

```

games.away_team_id AS team_id,
games.away_score AS runs_scored,
games.home_score AS runs_allowed,
CASE
  WHEN games.is_home_team_win = false THEN 1
  ELSE 0
END AS is_win
FROM
games;

```

How the Tables Interact with each other

The relational schema is connected by the games table, serving as the primary table correlating teams, parks, and umpires. Franchise data is integrated into this central hub via home, away, and winning team variables, while the parks table facilitates the inclusion of structural stadium attributes such as elevation. Umpires are integrated through four distinct umpire identifiers, mapping specific assignments for each base and home plate to individual contest records.

Within the postgres, the starting lineups table establishes the relationship between athletes and franchises for discrete games, documenting batting order slots and defensive mapping. The statcast_pitches table operates as an extensive ledger where each entry documents a single pitch delivery; this component utilizes batter and pitcher identifiers to maintain direct relational links with the players registry. Roster management and organizational transitions are managed by the players and transactions tables, of which utilizes franchise identifiers to monitor athlete trades and injury status across the league.

The database architecture employs specialized PostgreSQL views to transform raw ingestion streams into analysis-ready feature layers. At the franchise level, view_team_games normalizes the schedule index into a singular row per team format, enabling view_team_features to calculate rolling performance windows for victories and runs scored. For athletic performance, view_bullpen_usage and view_pitch_arsenal aggregate granular telemetry to quantify workloads and pitch metrics, while view_player_features applies retrospective windows to monitor recent pitching volume. Ultimately, the view_prediction_dataset functions as the definitive assembly framework for modeling, consolidating the game registry, venue elevation constants, and rolling metrics for both the home and away teams to create a clean modeling dataset with a binary win-loss target.

Ingestion of Data

To ensure the understanding of the update mechanisms within the data warehouse, it is essential to distinguish between the two core functional frameworks, the Historical Backfill Engine and the Daily Runner Production Pipeline. Although both infrastructures utilize the official MLB Stats API as the authoritative upstream source, they differ significantly in their temporal scope and computational resource allocation.

Upstream Architecture: MLB Stats API

Serving as the repository for Major League Baseball performance metrics, the MLB Stats API functions as a publicly accessible interface. While the system provides read only access without requiring authentication, strict adherence to URL formatting protocols is mandatory. The current implementation leverages three primary API components:

Schedule Index (/api/v1/schedule)

Overview: A relational directory correlating calendar dimensions with specific game identifiers.

Utility: By transmitting date parameters, the pipeline extracts contest logs, including unique gamePk constants, venue mappings, participating franchises, and finalized outcomes.

Live Data Feed (/api/v1.1/game/{gamePk}/feed/live)

Overview: A high frequency stream capturing granular performance data for every pitch delivered during a contest.

Utility: For each targeted gamePk, this endpoint yields a JSON payload encompassing Statcast metrics such as velocity, spin characteristics, and movement, which are funneled into the statcast_pitches table.

Transactions Wire (/api/v1/transactions)

Overview: A centralized ledger documenting all official franchise roster movements and league filings.

Utility: This component provides critical visibility into roster modifications including Injured List transitions and call-ups to support the team depth and bullpen fatigue.

Historical Backfill Methodology (backfill.py)

The Backfill Engine operates as an acquisition utility, primarily utilized to populate the historical training window required for robust model development.

Operational Logic:

Temporal Boundaries: Explicit start and end parameters are manually defined to establish the historical collection window.

Sequential Iteration: The system initializes a chronological array, processing each date in successive order to maintain integrity.

Automated Ingestion: Core scraping scripts target finalized contests, bypassing active or deferred games to ensure a consistent data state.

Throughput Performance: The engine operates at maximum API capacity to efficiently transfer historical information into the relational warehouse.

Daily Production Pipeline (run_daily_pipeline.py)

Upon the successful completion of the backfill phase, the Daily Runner assumes responsibility for ongoing warehouse maintenance, executing via automated cron jobs to ensure data currency at 4am each for the previous day game information.

Operational Logic:

Dynamic Scheduling: This engine utilizes system time to establish a 24-hour lookback period, automatically targeting the preceding day’s performance metrics.

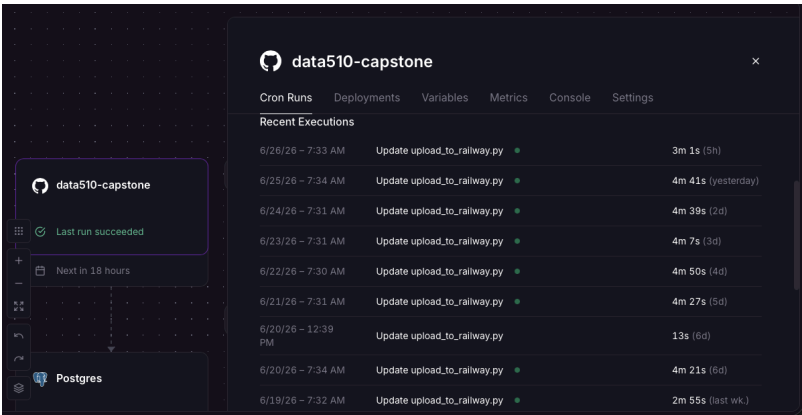
Incremental Updates: The pipeline extracts only specific data points from the targeted date, integrating new games and roster actions into the production environment.

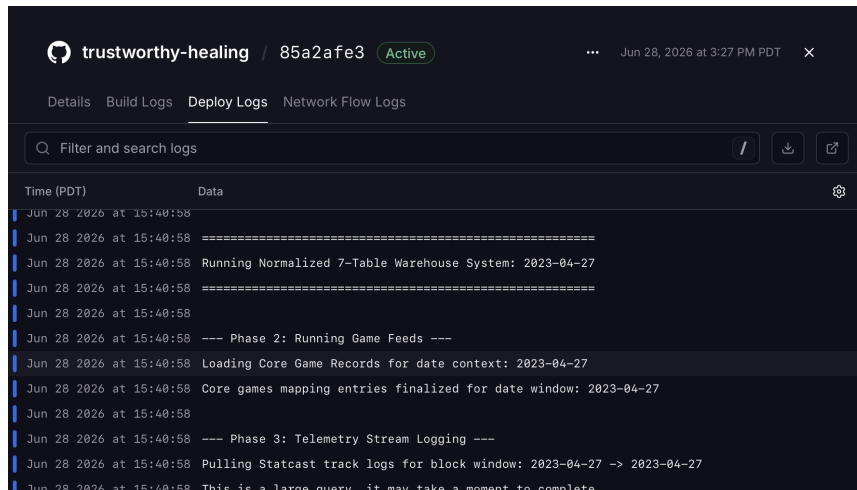
View Synchronization: Employing an UPSERT, the runner ensures that analytical feature layers, such as rolling performance averages, are updated to reflect scoring adjustments.

Architectural Synergy

The Backfill and Daily Runner infrastructures represent integrated components of a singular data engine. Both architectures leverage identical underlying scraping utilities, differing solely in the chronological parameters provided during execution. While the Backfill Engine provides the extensive historical memory required for robust model training, the Daily Runner ensures the ongoing daily collection for the dataset.

Evidence





Data Quality and Pulling

In summary, the ingestion pipelines functioned effectively, successfully backfilling all historical parameters. Within the dynamic environment of Major League Baseball, factors such as rookie debuts and active roster transitions introduce specific variables into the data architecture.

Statcast Pitch Metric

The Statcast pitch metrics table may contain null entries, this occurrence is not indicative of system failure, but rather a reflection of instances where a pitch does not result in a recorded hit or walk. Consequently, these missing values are expected and do not compromise the integrity of the dataset or its utility for future modeling phases.

Environmental Data for Indoor Venues

For games played in doomed stadiums, the source API often yields inconsistent text strings like "Wind: 0 mph, Roof Closed". To maintain consistency, a regular expression parser intercepts these instances, normalizing wind_speed_mph to 0 and assigning a categorical 'INDOORS' value to the wind_direction field, thereby preventing modeling skews.

Character Limit Constraints

International venues occasionally return lengthy localized name strings that exceed database character limits. The system utilizes proactive slicing to ensure compliance with the pre-defined relational schema.

New Player Profile Management

Mid season roster additions are frequently missing from initial pre-season data. Upon identifying an unmapped athlete, the scraper initiates a just-in-time request to the specific biography endpoint to maintain strict relational data integrity.

Raw and Analytical Data

To ensure relational consistency, initial data streams are directed into a normalized database architecture. This framework supports the automated maintenance of analysis-ready feature layers through the utilization of server side PostgreSQL Views.

Reproductibility

Access to the data collection infrastructure is facilitated by cloning the repository and establishing a Railway environment. To initialize the relational framework, users must obtain the *schema.sql* file from the github repository, which contains all necessary table and view definitions for the analysis. Given that the primary scraping utility focuses on the preceding day's contests, an additional GitHub repository must be integrated within Railway to maintain connectivity. It is important to note that the initial ingestion process requires substantial processing time, as it systematically extracts performance metrics across multiple seasons. During the environment configuration, ensuring the correct URL connection variable is mandatory for successful integration. Furthermore, users must verify that storage capacity is available to house the resulting dataset. To ensure a comprehensive data refresh, the backfill parameters must be manually adjusted to the desired historical endpoint, failure to define these date ranges or to ensure all tables are pre-configured may result in incomplete data acquisition or unexpected system behavior.

Ethics and Access Control

PII Risk Analysis

Within the scope of this research, it is confirmed that no restricted Personally Identifiable Information including sensitive athlete financial documentation, private health histories, or confidential contact records is contained within the public Major League Baseball data engine. The dataset is composed exclusively of publicly accessible athletic profiles, encompassing professional performance metrics such as physical dimensions, handedness, and aggregate batting statistics.

Data Privacy

Integrity of the acquired information is maintained through storage on a secured, private database server. All internal communication requires encrypted strings to ensure security, and administrative access is strictly limited to authorized model production environments to prevent unauthorized exposure.

Freeze Statement

At present, there are no further plans or expectations to modify or expand the data collection pipelines within the current scope, as all comprehensive data has been successfully acquired. Ongoing data collection will be limited to automated cron jobs, which are scheduled to run daily to capture information from the preceding day.